

Despliegue de seguridad en 2 droplets DigitalOcean — Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Traer la seguridad de la rama `seguridad_api` a `master` y producir todos los artefactos de infraestructura (docker-compose, HAProxy, nginx, Prometheus, Grafana, UFW) necesarios para separar el despliegue en 2 droplets DigitalOcean (privado con datos/lógica, público con borde/monitoreo) comunicados por VPC, cumpliendo la rúbrica completa de seguridad.

Architecture: Ver `docs/superpowers/specs/2026-07-31-two-droplet-security-deploy-design.md`. Droplet privado (existente, 2GB, IP VPC `.101`): postgres + api1/api2 (FastAPI) + flask1/flask2 (web1) + prometheus/cadvisor/node_exporter/postgres_exporter. Droplet público (nuevo, IP VPC `.102`): HAProxy (SSL + balanceo + stats) + nginx (portal estático) + Grafana.

Tech Stack: FastAPI, Flask, PostgreSQL 15, Docker Compose, HAProxy 3.2, nginx:alpine, Prometheus, Grafana, cadvisor, node_exporter, postgres_exporter, UFW.

Global Constraints

- Solo IP pública + certificado SSL autofirmado — sin dominio (decisión ya tomada).
- No se balancea PostgreSQL — una sola instancia.
- No se usa Redis ni mysql_exporter — no aplican al stack de SWAY (PostgreSQL, sin Redis).
- No se implementa CA propia — un solo cert autofirmado, sin jerarquía.
- El droplet privado reutiliza el droplet actual (`165.232.146.240`, `s-1vcpu-2gb-sfo3-01`) — no hay migración de datos.
- Todas las claves/secrets van por variable de entorno, nunca hardcodeadas en código Python (excepción documentada: `API_KEY` sí queda como literal en los clientes JS/mobile porque son públicos por naturaleza — ver Tarea 3-5).

Contexto técnico descubierto durante la investigación (no está en el spec, hace falta para ejecutar el plan)

- La rama `seguridad_api` aplica `Security(require_api_key)` **globalmente a todos los routers** en `app/main.py`. Esto significa que, tras el merge, **todo fetch existente en web1** (`assets/js/*.js`), **web2** (`web2/src/api/client.js`) y **mobile** (`MockupsSwayMobile/src/api/client.js`) **empezará a fallar con 401** si no manda el header `x-api-key`. El spec no lo menciona explícitamente pero es una consecuencia directa y bloqueante del merge — las Tareas 3, 4 y 5 de este plan lo resuelven.
- `docker-compose.prod.yml` (no `docker-compose.yml`, que es solo para desarrollo local) es el compose real de producción hoy: 1 `postgres`, 1 `api` (uvicorn, 2 workers), 1 `web` (unicorn Flask, 2

workers), 1 `nginx` sirviendo `/portal/` (volumen `web2/dist`), `/static/` (volumen `assets/`), y haciendo `proxy_pass` de `/api/`, `/docs/`, `/openapi.json` al contenedor `api`, y `/` al contenedor `web`.

- `Dockerfile` construye una sola imagen reusada tanto para `api` (`uvicorn app.main:app`) como para `web` (`gunicorn web:app`) — el mismo patrón se reutiliza para `api1/api2` y `flask1/flask2` en el nuevo `docker-compose.private.yml`.
 - No existe endpoint `/health` en `app/main.py` — hace falta agregarlo (Tarea 2) porque HAProxy lo necesita para el healthcheck de `api_back`.
 - Convención de tests del repo: `pytest` (ya instalado), archivos en `test/`, sin fixtures/clases — funciones sueltas que instancian el cliente y hacen asserts directos (ver `test/test_home.py`).
 - `web2/src/api/client.js` tiene un único punto central de fetch (`request()`) más un segundo fetch aislado para la descarga de PDF (`downloadReportePDF`) — 2 puntos de edición exactos, no hace falta refactor.
 - `MockupsSwayMobile/src/api/client.js` tiene 29 llamadas a `fetch()` dispersas en el archivo, sin punto central — la Tarea 5 usa un wrapper interno + reemplazo mecánico de `fetch()` por `apiFetch()` en vez de editar cada call site.
 - `assets/js/*.js` (web1) tiene fetches dispersos en 6+ archivos activos (`especies.js`, `eventos.js`, `main.js`, `mis-pedidos.js`, `tienda.js`) más archivos `_backup/` `_temp` que no se cargan en ningún template (dead code preexistente, no tocar). La Tarea 3 usa un monkey-patch de `window.fetch` en `main.js` (que ya se carga en todas las páginas) en vez de tocar cada archivo.
-

Task 1: Merge de seguridad_api a master (código de aplicación) + poda de artefactos VirtualBox

Files:

- Merge: rama `seguridad_api` → `master`
- Delete después del merge: `docker-compose.public.yml`, `docker-compose.private.yml` (versiones VirtualBox, se reescriben en Tareas 6-7), `nginx/nginx.conf`, `nginx/certs/` si vino algo, `scripts/ufw_public.sh`, `scripts/ufw_private.sh` (se reescriben en Tarea 13), `docs/handoff_claude_web.md`
- Keep tal cual del merge: `app/security/api_key.py`, `app/security/auth.py`, `app/security/rate_limit.py`, cambios en `app/routers/auth.py`, `app/routers/colaboradores.py`, `app/routers/pedidos.py`, `app/models/pedidos.py`, cambios de `app/main.py` (wiring de `SlowAPIMiddleware`, `custom_openapi`, `Security(require_api_key)` global), `requirements.txt` (línea `slowapi`), `.gitignore` (línea `nginx/certs/` — puede quedar aunque no haya carpeta `nginx/` ya, es inofensiva)
- Rename: `docs/seguridad_api.md` → `docs/seguridad_api_app_layer.md` (se mantiene como referencia histórica de la implementación de `bcrypt`/`JWT`/`API-key`/`rate-limit`/`BOLA-IDOR`, pero su sección 4 en adelante — arquitectura VirtualBox — queda obsoleta; agregar una nota al inicio del archivo aclarándolo)

Interfaces:

- Produce: `require_api_key` (`app/security/api_key.py`, dependencia `FastAPI`), `create_token/SECRET_KEY/ALGORITHM` (`app/security/auth.py`), `limiter` (`app/security/rate_limit.py`) — usados sin cambios por el resto del plan.

- ☐ **Step 1: Traer la rama y hacer el merge**

```
git fetch origin
git checkout master
git merge --no-ff seguridad_api -m "merge: seguridad_api - bcrypt, JWT, API
key, rate limiting, BOLA/IDOR"
```

Expected: merge limpio, 0 conflictos (ya verificado con `git merge-tree` en el spec).

- ☐ **Step 2: Verificar que el merge no rompió el arranque local**

```
pip install -r requirements.txt
python3 -c "import secrets; print(secrets.token_hex(32))" # generar
JWT_SECRET_KEY para .env local
python3 -c "import secrets; print(secrets.token_hex(24))" # generar API_KEY
para .env local
```

Agregar al `.env` local (no versionado):

```
JWT_SECRET_KEY=<valor generado>
API_KEY=<valor generado>
```

```
uvicorn app.main:app --port 8000
```

Expected: arranca sin traceback. En otra terminal:

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/api/estadisticas
```

Expected: `401` (antes del merge daba `200` — confirma que la API key ya se exige globalmente).

```
curl -s -o /dev/null -w "%{http_code}\n" -H "x-api-key: <valor de API_KEY del
.env>" http://localhost:8000/api/estadisticas
```

Expected: `200`.

- ☐ **Step 3: Podar artefactos VirtualBox y renombrar doc histórica**

```
git rm -r --ignore-unmatch docker-compose.public.yml docker-compose.private.yml
nginx/nginx.conf scripts/ufw_public.sh scripts/ufw_private.sh
```

```
docs/handoff_claude_web.md
git mv docs/seguridad_api.md docs/seguridad_api_app_layer.md
```

Editar la primera línea de `docs/seguridad_api_app_layer.md` para anteponer:

```
> **Nota (2026-07-31):** Este documento describe la implementación original de
seguridad
> de aplicación (bcrypt, JWT, API key, rate limiting, BOLA/IDOR) y una primera
propuesta de
> arquitectura de 2 servidores sobre VirtualBox que **no se usó**. La
arquitectura real de
> despliegue (2 droplets DigitalOcean) está en
> `docs/superpowers/specs/2026-07-31-two-droplet-security-deploy-design.md`. La
parte de
> seguridad de aplicación (secciones 1-3) sigue vigente.
```

- ☐ **Step 4: Commit**

```
git add -A
git commit -m "chore: podar artefactos de despliegue VirtualBox tras merge de
seguridad_api"
```

Task 2: Endpoint `/health` para healthchecks de HAProxy

Interfaces:

- Produce: `GET /health` → `{"status": "ok"}`, sin autenticación (no pasa por `require_api_key`, definido directo en `app`, igual que `/`).
- ☐ **Step 1: Escribir el test que falla**

Crear `test/test_health_endpoint.py`:

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health_returns_200_without_api_key():
    """El healthcheck de HAProxy no debe requerir x-api-key."""
    resp = client.get("/health")
    assert resp.status_code == 200

def test_health_body_is_status_ok():
```

```
resp = client.get("/health")
assert resp.json() == {"status": "ok"}
```

- ☐ **Step 2: Correr el test y confirmar que falla**

```
pytest test/test_health_endpoint.py -v
```

Expected: **FAIL** — **404 Not Found** (la ruta no existe todavía).

- ☐ **Step 3: Implementar el endpoint**

En `app/main.py`, después de la función `root()` existente:

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

- ☐ **Step 4: Correr el test y confirmar que pasa**

```
pytest test/test_health_endpoint.py -v
```

Expected: **2 passed**.

- ☐ **Step 5: Commit**

```
git add app/main.py test/test_health_endpoint.py
git commit -m "feat: agregar endpoint /health sin autenticación para
healthcheck de HAProxy"
```

Task 3: Web1 (`assets/js/main.js`) — inyectar `x-api-key` en todos los fetch

Files:

- Modify: `assets/js/main.js`

Interfaces:

- Consumes: nada de tareas anteriores (es JS de browser, independiente del backend Python).
- Produces: `window.fetch` parcheado — todas las llamadas existentes en `especies.js`, `eventos.js`, `mis-pedidos.js`, `tienda.js` (que llaman a `fetch(...)` tal cual, sin cambios) quedan cubiertas automáticamente porque `main.js` se carga antes que ellos en cada template.

- ☐ **Step 1: Confirmar que `main.js` se carga antes que los demás scripts de API**

```
grep -n "main.js\\|especies.js\\|eventos.js\\|tienda.js\\|mis-pedidos.js"
templates/*.html
```

Expected: en cada template que usa esos scripts, `main.js` aparece en un `<script>` anterior en el HTML (si algún template no lo carga, agregar `<script src="/static/js/main.js"></script>` antes de los demás — verificar caso por caso).

- ☐ **Step 2: Agregar el parche de fetch al final de `assets/js/main.js`**

```
// =====
// API KEY – inyección automática en todas las llamadas a la API
// La rama de seguridad exige x-api-key en todos los endpoints de FastAPI.
// Se parchea window.fetch una sola vez acá en vez de editar cada archivo
// que llama a la API (especies.js, eventos.js, tienda.js, mis-pedidos.js).
// =====
(function () {
  const SWAY_API_KEY = 'REEMPLAZAR_CON_API_KEY_PUBLICA';
  const originalFetch = window.fetch;
  window.fetch = function (input, init = {}) {
    const url = typeof input === 'string' ? input : (input && input.url) || '';
    if (url.includes('/api/')) {
      init = { ...init, headers: { ...(init.headers || {}), 'x-api-key':
SWAY_API_KEY } };
    }
    return originalFetch(input, init);
  };
})();
```

- ☐ **Step 3: Verificar manualmente contra el servidor local**

```
uvicorn app.main:app --port 8000
python3 app.py # Flask en :5000, otra terminal
```

Abrir <http://localhost:5000/especies> en el navegador, DevTools → Network → confirmar que las peticiones a `/api/especies` llevan el header `x-api-key` y responden `200` (no `401`).

- ☐ **Step 4: Commit**

```
git add assets/js/main.js
git commit -m "fix(web1): inyectar x-api-key en todas las llamadas a la API vía
patch de window.fetch"
```

Task 4: Web2 (`web2/src/api/client.js`) — agregar `x-api-key`

Files:

- Modify: `web2/src/api/client.js`
- ☐ **Step 1: Agregar la constante y editar `request()`**

Al inicio del archivo, después de `const BASE = '/api':`

```
const API_KEY = 'REEMPLAZAR_CON_API_KEY_PUBLICA'
```

Dentro de `request()`, reemplazar:

```
const headers = {}  
if (body) headers['Content-Type'] = 'application/json'  
if (token) headers['Authorization'] = `Bearer ${token}`
```

por:

```
const headers = { 'x-api-key': API_KEY }  
if (body) headers['Content-Type'] = 'application/json'  
if (token) headers['Authorization'] = `Bearer ${token}`
```

- ☐ **Step 2: Editar `downloadReportePDF`**

Reemplazar:

```
const headers = {}  
if (token) headers['Authorization'] = `Bearer ${token}`
```

por:

```
const headers = { 'x-api-key': API_KEY }  
if (token) headers['Authorization'] = `Bearer ${token}`
```

(dentro de `downloadReportePDF`, no de `request()` — son dos bloques `const headers = {}` distintos en el archivo).

- ☐ **Step 3: Verificar manualmente**

```
uvicorn app.main:app --port 8000
cd web2 && npm run dev
```

Abrir <http://localhost:5173>, login como colaborador, confirmar en Network que las peticiones a </api/...> llevan `x-api-key` y responden `200`. Probar también "Descargar Reporte PDF" — debe descargar el archivo sin `401`.

- ☐ **Step 4: Commit**

```
git add web2/src/api/client.js
git commit -m "fix(web2): agregar x-api-key en request() y downloadReportePDF"
```

Task 5: Mobile ([MockupsSwayMobile/src/api/client.js](#)) — inyectar `x-api-key`

Files:

- Modify: [MockupsSwayMobile/src/api/client.js](#)

Interfaces:

- Produces: `apiFetch(url, opts)` — reemplaza todos los usos internos de `fetch()` en este archivo. No cambia ninguna firma exportada (`login`, `getEspecies`, etc. siguen igual para quien los consume desde las pantallas).
- ☐ **Step 1: Agregar la constante y el wrapper cerca del inicio del archivo**

Después de la constante `export const API_HOST = 'http://165.232.146.240';`:

```
const SWAY_API_KEY = 'REEMPLAZAR_CON_API_KEY_PUBLICA';

function apiFetch(url, opts = {}) {
  return fetch(url, { ...opts, headers: { ...(opts.headers || {}), 'x-api-key': SWAY_API_KEY } });
}
```

- ☐ **Step 2: Reemplazo mecánico de todas las llamadas**

Reemplazar **todas** las ocurrencias de `fetch()` por `apiFetch()` en el resto del archivo, **excepto** la que queda dentro de la definición de `apiFetch` misma (esa debe seguir siendo `fetch()` real, si no, recursión infinita).

```
grep -c "fetch(" MockupsSwayMobile/src/api/client.js
```

Anotar el número (debe bajar en 1 respecto al original tras el reemplazo — la única `fetch()` real que queda es la de dentro de `apiFetch`).

- ☐ **Step 3: Verificar manualmente contra el droplet actual**

```
cd MockupsSwayMobile && npx expo start
```

Abrir la app en Expo Go (conectado al droplet vía `API_HOST` ya hardcodedo), hacer login, ver Especies/Avistamientos/Eventos. Confirmar que nada devuelve error de sesión/conexión inesperado (una vez que el droplet tenga la `API_KEY` configurada en su `.env` — esto se prueba en conjunto con la Tarea 15 de despliegue real, acá solo se verifica que el código compila y ejecuta sin excepciones de sintaxis).

```
node -e "require('@babel/core')" 2>/dev/null; npx eslint src/api/client.js
```

Expected: sin errores de sintaxis.

- ☐ **Step 4: Commit**

```
git add MockupsSwayMobile/src/api/client.js
git commit -m "fix(mobile): inyectar x-api-key vía wrapper apiFetch en todas las llamadas"
```

Task 6: `docker-compose.private.yml` (droplet privado)

Files:

- Create: `docker-compose.private.yml`

Interfaces:

- Consumes: `Dockerfile` existente (build reusado para `api1/api2/flask1/flask2`), `SWAY_PostgreSQL.sql` existente (init de BD).
- Produces: servicios `postgres`, `api1`, `api2`, `flask1`, `flask2`, `prometheus`, `cadvisor`, `node_exporter`, `postgres_exporter` — nombres de servicio consumidos por Tarea 8 (`haproxy.cfg`) y Tarea 10 (`prometheus.yml`).

- ☐ **Step 1: Crear el archivo**

```
services:
  postgres:
    image: postgres:15
    container_name: sway_postgres
    restart: unless-stopped
```

```

environment:
  POSTGRES_USER: ${DB_USER:-sway_app}
  POSTGRES_PASSWORD: ${DB_PASSWORD:-sway123}
  POSTGRES_DB: ${DB_NAME:-sway}
volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./SWAY_PostgreSQL.sql:/docker-entrypoint-initdb.d/01_init.sql
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-sway_app} -d ${DB_NAME:-sway}"]
  interval: 5s
  retries: 5
  timeout: 5s
networks:
  - data_network

api1:
  build: .
  container_name: sway_api1
  restart: unless-stopped
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000
  env_file:
    - path: .env
      required: false
  environment:
    DATABASE_URL: postgresql+psycopg://${DB_USER:-sway_app}:${DB_PASSWORD:-sway123}@postgres:5432/${DB_NAME:-sway}
    CORS_ORIGINS: ${CORS_ORIGINS:-http://localhost:5000,http://localhost:5173}
  ports:
    - "8001:8000"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - app_network
    - data_network

api2:
  build: .
  container_name: sway_api2
  restart: unless-stopped
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000
  env_file:
    - path: .env
      required: false
  environment:
    DATABASE_URL: postgresql+psycopg://${DB_USER:-sway_app}:${DB_PASSWORD:-sway123}@postgres:5432/${DB_NAME:-sway}
    CORS_ORIGINS: ${CORS_ORIGINS:-http://localhost:5000,http://localhost:5173}
  ports:
    - "8002:8000"

```

```

depends_on:
  postgres:
    condition: service_healthy
networks:
  - app_network
  - data_network

flask1:
  build: .
  container_name: sway_flask1
  restart: unless-stopped
  command: gunicorn web:app --bind 0.0.0.0:5000 --workers 2 --timeout 120
  env_file:
    - path: .env
      required: false
  environment:
    DATABASE_URL: postgresql+psycopg://${DB_USER:-sway_app}:${DB_PASSWORD:-
sway123}@postgres:5432/${DB_NAME:-sway}
    SECRET_KEY: ${SECRET_KEY:-sway_secret_key_ultra_secret}
    DEBUG: "False"
    FLASK_ENV: production
    API_BASE_URL: /api
  ports:
    - "5001:5000"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - app_network
    - data_network

flask2:
  build: .
  container_name: sway_flask2
  restart: unless-stopped
  command: gunicorn web:app --bind 0.0.0.0:5000 --workers 2 --timeout 120
  env_file:
    - path: .env
      required: false
  environment:
    DATABASE_URL: postgresql+psycopg://${DB_USER:-sway_app}:${DB_PASSWORD:-
sway123}@postgres:5432/${DB_NAME:-sway}
    SECRET_KEY: ${SECRET_KEY:-sway_secret_key_ultra_secret}
    DEBUG: "False"
    FLASK_ENV: production
    API_BASE_URL: /api
  ports:
    - "5002:5000"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - app_network

```

```

    - data_network

postgres_exporter:
  image: prometheuscommunity/postgres-exporter:latest
  container_name: sway_postgres_exporter
  restart: unless-stopped
  environment:
    DATA_SOURCE_NAME: postgresql://${DB_USER:-sway_app}:${DB_PASSWORD:-
sway123}@postgres:5432/${DB_NAME:-sway}?sslmode=disable
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - data_network
    - monitoring_network

cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: sway_cadvisor
  restart: unless-stopped
  privileged: true
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
  networks:
    - monitoring_network

node_exporter:
  image: prom/node-exporter:latest
  container_name: sway_node_exporter
  restart: unless-stopped
  command:
    - --path.rootfs=/host
  volumes:
    - /:/host:ro,rslave
  networks:
    - monitoring_network

prometheus:
  image: prom/prometheus:latest
  container_name: sway_prometheus
  restart: unless-stopped
  command:
    - --config.file=/etc/prometheus/prometheus.yml
    - --storage.tsdb.path=/prometheus
    - --storage.tsdb.retention.time=15d
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus_data:/prometheus
  ports:
    - "9090:9090"

```

```

networks:
  - app_network
  - monitoring_network

networks:
  app_network:
  data_network:
    internal: true
  monitoring_network:
    internal: true

volumes:
  postgres_data:
  prometheus_data:

```

- ☐ **Step 2: Validar sintaxis**

```
docker compose -f docker-compose.private.yml config --quiet
```

Expected: sin salida (sin errores). Si falta `.env`, exportar variables mínimas antes: `export DB_USER=sway_app DB_PASSWORD=sway123 DB_NAME=sway` — el comando `config` no requiere que los contenedores levanten, solo valida el YAML.

- ☐ **Step 3: Commit**

```

git add docker-compose.private.yml
git commit -m "feat: docker-compose.private.yml – postgres + api1/api2 +
flask1/flask2 + stack de métricas para droplet privado"

```

Task 7: `docker-compose.public.yml` (droplet público)

Files:

- Create: `docker-compose.public.yml`
- Create: `haproxy/certs/.gitkeep` (carpeta vacía versionada, el cert real se genera en la Tarea 12 y no se versiona)
- Modify: `.gitignore` — agregar `haproxy/certs/*.pem` y `haproxy/certs/*.key`

Interfaces:

- Consumes: `haproxy/haproxy.cfg` (Tarea 8), `nginx/portal.conf` (Tarea 9), `haproxy/certs/server.pem` (Tarea 12, generado en el droplet, no versionado).
- Produces: servicios `haproxy`, `nginx-portal`, `grafana`.

- ☐ **Step 1: Crear el archivo**

```

services:
  haproxy:
    image: haproxy:3.2-alpine
    container_name: sway_haproxy
    restart: unless-stopped
    volumes:
      - ./haproxy/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
      - ./haproxy/certs:/usr/local/etc/haproxy/certs:ro
    ports:
      - "80:80"
      - "443:443"
      - "8404:8404"
      - "8405:8405"
    depends_on:
      - nginx-portal
      - grafana
    networks:
      - edge_network

  nginx-portal:
    image: nginx:alpine
    container_name: sway_nginx_portal
    restart: unless-stopped
    volumes:
      - ./web2/dist:/usr/share/nginx/html/portal:ro
      - ./assets:/usr/share/nginx/html/static:ro
      - ./nginx/portal.conf:/etc/nginx/conf.d/default.conf:ro
    expose:
      - "80"
    networks:
      - edge_network

  grafana:
    image: grafana/grafana:latest
    container_name: sway_grafana
    restart: unless-stopped
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:-changeme}
      GF_USERS_ALLOW_SIGN_UP: "false"
      GF_SERVER_ROOT_URL: "%(protocol)s://%(domain)s/grafana/"
      GF_SERVER_SERVE_FROM_SUB_PATH: "true"
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    expose:
      - "3000"
    networks:
      - edge_network

networks:

```

```
edge_network:

volumes:
  grafana_data:
```

- ☐ **Step 2: Crear placeholder de certs y actualizar .gitignore**

```
mkdir -p haproxy/certs
touch haproxy/certs/.gitkeep
```

Agregar a **.gitignore**:

```
haproxy/certs/*.pem
haproxy/certs/*.key
haproxy/certs/*.crt
!haproxy/certs/.gitkeep
```

- ☐ **Step 3: Validar sintaxis (fallará hasta Tareas 8-9, es esperado)**

```
docker compose -f docker-compose.public.yml config --quiet
```

Expected: sin errores de YAML (los **volumes** que apuntan a **haproxy/haproxy.cfg** y **nginx/portal.conf** no necesitan existir todavía para que **config** valide — solo valida sintaxis del compose, no que los archivos montados existan).

- ☐ **Step 4: Commit**

```
git add docker-compose.public.yml haproxy/certs/.gitkeep .gitignore
git commit -m "feat: docker-compose.public.yml - HAProxy + nginx-portal +
Grafana para droplet público"
```

Task 8: **haproxy/haproxy.cfg**

Files:

- Create: **haproxy/haproxy.cfg**

Interfaces:

- Consumes: nombres de servicio **nginx-portal:80**, **grafana:3000** (Tarea 7, red **edge_network**); IPs privadas VPC de los backends del droplet privado (placeholders **IP_PRIVADA_VPC**, se rellenan en la Tarea 15 con la IP real).

- Produces: :80, :443, :8404 (/stats), :8405 (/metrics) — consumidos por Tarea 10 (prometheus.yml scrapea :8405) y por el runbook de despliegue (Tarea 15).
- ☐ **Step 1: Crear el archivo**

```
global
    log stdout format raw local0
    maxconn 4096
    ssl-default-bind-options ssl-min-ver TLSv1.2

defaults
    log global
    mode http
    option httplog
    option forwardfor

    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend http_front
    bind *:80
    http-request redirect scheme https code 301

frontend https_front
    bind *:443 ssl crt /usr/local/etc/haproxy/certs/server.pem

    acl path_api      path_beg /api
    acl path_docs     path_beg /docs
    acl path_openapi  path_beg /openapi.json
    acl path_portal    path_beg /portal
    acl path_static    path_beg /static
    acl path_grafana   path_beg /grafana

    use_backend api_back      if path_api or path_docs or path_openapi
    use_backend portal_back   if path_portal or path_static
    use_backend grafana_back  if path_grafana

    default_backend flask_back

backend api_back
    balance roundrobin
    option httpchk GET /health
    http-check expect status 200
    server api1 IP_PRIVADA_VPC:8001 check
    server api2 IP_PRIVADA_VPC:8002 check

backend flask_back
    balance roundrobin
    option httpchk GET /
    http-check expect rstatus (2|3)[0-9][0-9]
```

```

server flask1 IP_PRIVADA_VPC:5001 check
server flask2 IP_PRIVADA_VPC:5002 check

backend portal_back
    server portal nginx-portal:80 check

backend grafana_back
    server grafana grafana:3000 check

listen stats
    bind *:8404
    mode http
    stats enable
    stats uri /stats
    stats refresh 5s
    stats show-legends
    stats show-node

frontend prometheus_front
    bind *:8405
    mode http
    http-request use-service prometheus-exporter if { path /metrics }
    http-request return status 404

```

- ☐ **Step 2: Validar sintaxis con el binario de HAProxy en Docker (no requiere certs reales todavía)**

```

mkdir -p /tmp/haproxy-check-certs
openssl req -x509 -newkey rsa:2048 -keyout /tmp/haproxy-check-certs/server.key
-out /tmp/haproxy-check-certs/server.crt -days 1 -nodes -subj "/CN=check"
cat /tmp/haproxy-check-certs/server.crt /tmp/haproxy-check-certs/server.key >
/tmp/haproxy-check-certs/server.pem
docker run --rm -v
"$ (pwd) /haproxy/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro" -v
"/tmp/haproxy-check-certs:/usr/local/etc/haproxy/certs:ro" haproxy:3.2-alpine
haproxy -c -f /usr/local/etc/haproxy/haproxy.cfg

```

Expected: Configuration file is valid.

- ☐ **Step 3: Commit**

```

git add haproxy/haproxy.cfg
git commit -m "feat: haproxy.cfg — SSL, balanceo roundrobin api/flask, stats,
exporter prometheus"

```

Task 9: **nginx/portal.conf** (nginx interno del droplet público)

Files:

- Create: `nginx/portal.conf`

Interfaces:

- Consumes: volúmenes montados en Tarea 7 (`web2/dist` en `/usr/share/nginx/html/portal`, `assets/` en `/usr/share/nginx/html/static`).
- Produce: puerto `80` interno (`expose`, no publicado), consumido por backend `portal_back` en `haproxy.cfg` (Tarea 8).
- ☐ **Step 1: Crear el archivo**

```
server {
    listen 80;
    client_max_body_size 10M;

    location /portal/ {
        alias /usr/share/nginx/html/portal/;
        try_files $uri $uri/ /portal/index.html;
        expires 1d;
        add_header Cache-Control "public";
    }

    location /portal/assets/ {
        alias /usr/share/nginx/html/portal/assets/;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location /static/ {
        alias /usr/share/nginx/html/static/;
        expires 7d;
        add_header Cache-Control "public";
    }
}
```

- ☐ **Step 2: Validar sintaxis**

```
docker run --rm -v "$(pwd)/nginx/portal.conf:/etc/nginx/conf.d/default.conf:ro"
nginx:alpine nginx -t
```

Expected: `syntax is ok / test is successful`.

- ☐ **Step 3: Commit**

```
git add nginx/portal.conf
git commit -m "feat: nginx/portal.conf – sirve web2/dist y assets estáticos de
Flask en el droplet público"
```

Task 10: `prometheus/prometheus.yml` (droplet privado)

Files:

- Create: `prometheus/prometheus.yml`

Interfaces:

- Consumes: nombres de servicio `cadvisor:8080`, `node_exporter:9100`, `postgres_exporter:9187` (Tarea 6, red `monitoring_network`); IP privada VPC del droplet público puerto `8405` (placeholder `IP_PUBLICA_VPC`, se rellena en Tarea 15).
- ☐ **Step 1: Crear el archivo**

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: [prometheus:9090]

  - job_name: cadvisor
    static_configs:
      - targets: [cadvisor:8080]

  - job_name: node
    static_configs:
      - targets: [node_exporter:9100]

  - job_name: postgres
    static_configs:
      - targets: [postgres_exporter:9187]

  - job_name: haproxy-edge
    metrics_path: /metrics
    static_configs:
      - targets: ["IP_PUBLICA_VPC:8405"]
```

- ☐ **Step 2: Validar sintaxis**

```
docker run --rm -v
"$(pwd)/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro"
prom/prometheus:latest promtool check config /etc/prometheus/prometheus.yml
```

Expected: **SUCCESS**.

- ☐ **Step 3: Commit**

```
git add prometheus/prometheus.yml
git commit -m "feat: prometheus.yml - scrape local (cadvisor/node/postgres) +
remoto (haproxy /metrics)"
```

Task 11: Grafana — datasource y dashboard de balanceo

Files:

- Create: grafana/provisioning/datasources/prometheus.yml
- Create: grafana/provisioning/dashboards/dashboards.yml
- Create: grafana/provisioning/dashboards/sway-balanceo.json

Interfaces:

- Consumes: IP privada VPC del droplet privado puerto 9090 (placeholder `IP_PRIVADA_VPC`, se rellena en Tarea 15); montado por Tarea 7 en `/etc/grafana/provisioning`.

- ☐ **Step 1: Datasource**

grafana/provisioning/datasources/prometheus.yml:

```
apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://IP_PRIVADA_VPC:9090
  isDefault: true
```

- ☐ **Step 2: Registro de dashboard**

grafana/provisioning/dashboards/dashboards.yml:

```
apiVersion: 1
providers:
- name: SWAY
  folder: SWAY
```

```
type: file
options:
  path: /etc/grafana/provisioning/dashboards
```

- ❑ **Step 3: Dashboard con panel de reparto de tráfico**

grafana/provisioning/dashboards/sway-balanceo.json:

```
{
  "title": "SWAY – Balanceo y Monitoreo",
  "timezone": "browser",
  "panels": [
    {
      "id": 1,
      "title": "Peticiones por backend (HAProxy)",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 24, "x": 0, "y": 0 },
      "targets": [
        {
          "expr": "sum by (proxy)
(rate(haproxy_server_http_responses_total[1m]))",
          "legendFormat": "{{proxy}}"
        }
      ]
    },
    {
      "id": 2,
      "title": "Backends activos (up/down)",
      "type": "stat",
      "gridPos": { "h": 6, "w": 12, "x": 0, "y": 8 },
      "targets": [
        { "expr": "haproxy_server_up" }
      ]
    },
    {
      "id": 3,
      "title": "Uso de CPU por contenedor",
      "type": "timeseries",
      "gridPos": { "h": 6, "w": 12, "x": 12, "y": 8 },
      "targets": [
        { "expr": "sum by (name)
(rate(container_cpu_usage_seconds_total{name!=\"\\\"\\\"}[1m]))" }
      ]
    }
  ],
  "schemaVersion": 39,
  "version": 1
}
```

- ☐ **Step 4: Validar JSON**

```
python3 -m json.tool grafana/provisioning/dashboards/sway-balanceo.json > /dev/null && echo "JSON válido"
```

- ☐ **Step 5: Commit**

```
git add grafana/provisioning
git commit -m "feat: provisioning de Grafana – datasource Prometheus + dashboard de balanceo HAProxy"
```

Task 12: Script de generación de certificado SSL autofirmado

Files:

- Create: `haproxy/generate_cert.sh`

Interfaces:

- Produce: `haproxy/certs/server.pem` (no versionado, generado en el droplet público al desplegar — Tarea 15 lo invoca).

- ☐ **Step 1: Crear el script**

```
#!/bin/bash
set -euo pipefail
cd "$(dirname "$0")/certs"

openssl req -x509 -newkey rsa:2048 \
  -keyout server.key \
  -out server.crt \
  -days 365 -nodes \
  -subj "/CN=sway.local/O=SWAY Conservacion Marina/C=MX"

cat server.crt server.key > server.pem
chmod 600 server.key server.pem

echo "Certificado generado en haproxy/certs/server.pem"
```

- ☐ **Step 2: Dar permisos de ejecución y probar localmente**

```
chmod +x haproxy/generate_cert.sh
./haproxy/generate_cert.sh
```

```
ls -la haproxy/certs/  
openssl x509 -in haproxy/certs/server.crt -noout -subject
```

Expected: `subject=CN = sway.local, O = SWAY Conservacion Marina, C = MX`. Luego borrar los archivos generados localmente (no se versionan, se regeneran en el droplet real):

```
rm haproxy/certs/server.key haproxy/certs/server.crt haproxy/certs/server.pem
```

- ☐ **Step 3: Commit**

```
git add haproxy/generate_cert.sh  
git commit -m "feat: script de generación de certificado SSL autofirmado para  
HAProxy"
```

Task 13: UFW — scripts de firewall por droplet

Files:

- Create: `scripts/ufw_private.sh`
- Create: `scripts/ufw_public.sh`

Interfaces:

- Consumes: IPs privadas VPC de ambos droplets (placeholders, rellenos en Tarea 15).
- ☐ **Step 1: `scripts/ufw_private.sh`**

```
#!/bin/bash  
set -euo pipefail  
  
ufw default deny incoming  
ufw default allow outgoing  
  
ufw allow 22/tcp  
  
# API y Flask balanceados – solo alcanzables desde el droplet público (VPC)  
ufw allow from IP_PUBLICA_VPC to any port 8001 proto tcp  
ufw allow from IP_PUBLICA_VPC to any port 8002 proto tcp  
ufw allow from IP_PUBLICA_VPC to any port 5001 proto tcp  
ufw allow from IP_PUBLICA_VPC to any port 5002 proto tcp  
  
# Prometheus (:9090) accesible solo desde el droplet público, para que Grafana  
# lo consulte directo por la VPC – no expuesto a internet.  
ufw allow from IP_PUBLICA_VPC to any port 9090 proto tcp
```

```
# postgres (5432) NO se abre – solo la red Docker interna lo alcanza

ufw --force enable
ufw status verbose
```

- ☐ **Step 2: scripts/ufw_public.sh**

```
#!/bin/bash
set -euo pipefail

ufw default deny incoming
ufw default allow outgoing

ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 8404/tcp

# Metrics de HAProxy (:8405) – solo el Prometheus del droplet privado lo
scrapea
ufw allow from IP_PRIVADA_VPC to any port 8405 proto tcp

ufw --force enable
ufw status verbose
```

- ☐ **Step 3: Dar permisos de ejecución**

```
chmod +x scripts/ufw_private.sh scripts/ufw_public.sh
```

- ☐ **Step 4: Commit**

```
git add scripts/ufw_private.sh scripts/ufw_public.sh
git commit -m "feat: scripts UFW diferenciados por droplet, restringidos a IPs
VPC"
```

Task 14: .env.example — variables nuevas

Files:

- Modify: .env.example
- ☐ **Step 1: Actualizar el archivo**

```
# Copiar como .env en el droplet PRIVADO y rellenar con valores reales
# chmod 600 .env

# PostgreSQL
DB_USER=sway_app
DB_PASSWORD=sway123
DB_NAME=sway

# Flask
SECRET_KEY=genera_con: python3 -c "import secrets;
print(secrets.token_hex(32))"

# JWT (FastAPI) – genera con: python3 -c "import secrets;
print(secrets.token_hex(32))"
JWT_SECRET_KEY=REEMPLAZAR_CON_CLAVE_HEX_64_CHARS

# API Key – genera con: python3 -c "import secrets;
print(secrets.token_hex(24))"
# Debe coincidir con la constante hardcodeada en assets/js/main.js,
web2/src/api/client.js
# y MockupsSwayMobile/src/api/client.js – es una clave pública anti-scraping,
no un secreto
# de alta seguridad (se distribuye a todos los clientes por diseño).
API_KEY=REEMPLAZAR_CON_CLAVE_HEX_48_CHARS

# Email (Gmail SMTP)
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USER=tu_correo@gmail.com
MAIL_PASS=tu_app_password_de_16_caracteres

# CORS – dominio/IP del droplet PÚBLICO (el que sirve HTTPS), sin slash final
CORS_ORIGINS=https://IP_DEL_DROPLET_PUBLICO

# Grafana admin (droplet público, va en el .env de ese droplet, no en este)
GRAFANA_ADMIN_PASSWORD=cambia_esto
```

- ☐ **Step 2: Commit**

```
git add .env.example
git commit -m "docs: actualizar .env.example con JWT_SECRET_KEY, API_KEY y
CORS_ORIGINS del nuevo split"
```

Task 15: Runbook de despliegue real (manual, ejecutado por el usuario sobre los droplets)

Files:

- Create: [docs/DEPLOYMENT_2_DROPLETS.md](#)

Esta tarea documenta los pasos que **el usuario ejecuta por SSH sobre los droplets reales** — no son ejecutables desde este entorno (sin acceso SSH/API de DigitalOcean). El documento debe ser lo bastante preciso para copiar/pegar sin adivinar nada.

- ☐ **Step 1: Escribir el runbook**

docs/DEPLOYMENT_2_DROPLETS.md:

```
# Despliegue en 2 droplets – Runbook
```

```
## 0. Prerrequisitos
```

- Droplet privado: el existente (`165.232.146.240`, `sway-server`, 2GB) – ya tiene el proyecto en `/home/sway/sway` con `master` actualizado (`git pull`).
- Crear droplet público nuevo en DigitalOcean: Ubuntu 22.04, mismo datacenter que el privado (mismo VPC), 1GB/\$6 alcanza (solo corre HAProxy+nginx+Grafana).
- En el panel de DigitalOcean, habilitar VPC networking para ambos droplets si no la tienen (mismo datacenter → misma VPC por defecto). Anotar la IP privada de cada uno (`ip addr show eth1` o el panel de Networking de DO).

```
## 1. Droplet privado – actualizar y aplicar UFW
```

```
```bash
ssh sway@165.232.146.240
cd /home/sway/sway
git pull
docker compose -f docker-compose.prod.yml down # baja el stack viejo de un
solo droplet
```

Reemplazar `IP_PUBLICA_VPC` en `scripts/ufw_private.sh` y `prometheus/prometheus.yml` (campo `haproxy-edge`) por la IP privada real del droplet público, luego:

```
sudo bash scripts/ufw_private.sh
docker compose -f docker-compose.private.yml up --build -d
docker compose -f docker-compose.private.yml ps
```

Verificar 9 contenedores **Up**: postgres, api1, api2, flask1, flask2, postgres\_exporter, cadvisor, node\_exporter, prometheus.

## 2. Droplet público — preparar y levantar

```
ssh root@<IP_PUBLICA>
adduser sway && usermod -aG sudo sway
rsync --archive --chown=sway:sway ~/.ssh /home/sway
su - sway
curl -fsSL https://get.docker.com | sudo bash
sudo usermod -aG docker sway
exit && ssh sway@<IP_PUBLICA>
```

```
git clone https://github.com/TU_USUARIO/TU_REPO.git sway
cd sway
```

Reemplazar `IP_PRIVADA_VPC` en `haproxy/haproxy.cfg` (backends `api_back/flask_back`) y en `grafana/provisioning/datasources/prometheus.yml` por la IP privada real del droplet privado. Reemplazar `IP_PUBLICA_VPC` en `scripts/ufw_public.sh` por la IP privada real del droplet privado (para permitir que su Prometheus scrapee :8405).

```
./haproxy/generate_cert.sh
mkdir -p web2/dist # o traer el build real: scp -r web2/dist desde tu máquina
cp .env.example .env
nano .env # GRAFANA_ADMIN_PASSWORD
sudo bash scripts/ufw_public.sh
docker compose -f docker-compose.public.yml up --build -d
docker compose -f docker-compose.public.yml ps
```

Verificar 3 contenedores Up: haproxy, nginx-portal, grafana.

### 3. Verificación end-to-end (desde cualquier máquina fuera de la VPC)

```
curl -k -s -o /dev/null -w "%{http_code}\n"
https://<IP_PUBLICA>/api/estadisticas -H "x-api-key: <API_KEY>" # 200
curl -k -s https://<IP_PUBLICA>/docs | grep -o "<title>.*</title>"
Swagger
curl -k -s -o /dev/null -w "%{http_code}\n" https://<IP_PUBLICA>/portal/
200
curl -k -s -o /dev/null -w "%{http_code}\n" https://<IP_PUBLICA>/
200, Flask
curl -k -s https://<IP_PUBLICA>:8404/stats | grep -o "<title>.*</title>"
stats HAProxy
curl -k -s https://<IP_PUBLICA>/grafana/login
Grafana
```

Desde el droplet privado, confirmar que las IPs directas no responden desde fuera de la VPC:

```
curl -m 3 http://<IP_PUBLICA_de_internet>:8001/ # debe fallar (UFW bloquea,
solo VPC pasa)
```

### 4. Prueba de balanceo visible (evidencia para la rúbrica)

```
for i in $(seq 1 20); do curl -sk https://<IP_PUBLICA>/api/estadisticas -H "x-
api-key: <API_KEY>" -o /dev/null; done
```

Abrir `https://<IP_PUBLICA>:8404/stats` — la fila `api1` y `api2` dentro de `api_back` deben mostrar sesiones/peticiones repartidas entre ambas, no todo en una. Mismo resultado esperado en el panel de Grafana "Peticiones por backend (HAProxy)".

## 5. Reconfigurar clientes con la nueva URL pública

- `MockupsSwayMobile/src/api/client.js`: `API_HOST` → `https://<IP_PUBLICA>`.
- `web2/vite.config.js` (proxy de dev) y build de producción: sin cambios si sigue siendo mismo-origen tras el build (`/api` relativo).
- `assets/js/main.js` / templates Flask: siguen usando rutas relativas `/api/...`, sin cambios — HAProxy ya enruta.

```
- [] **Step 2: Commit**
```

```
```bash
```

```
git add docs/DEPLOYMENT_2_DROPLETS.md
```

```
git commit -m "docs: runbook de despliegue manual en 2 droplets DigitalOcean"
```

Self-Review (ya aplicado al escribir el plan)

Cobertura del spec:

- Arquitectura 2 droplets → Tareas 6, 7, 15.
- HAProxy balanceo + stats + metrics → Tareas 8, 12.
- Prometheus + Grafana + cadvisor + node_exporter + postgres_exporter → Tareas 6, 10, 11.
- Firewall UFW → Tarea 13.
- SSL autofirmado → Tarea 12.
- JWT/API-key/bcrypt/rate-limit/BOLA-IDOR → Tarea 1 (merge, ya implementado en la rama).
- `/health` para HAProxy → Tarea 2.
- Consecuencia no cubierta por el spec pero bloqueante (API key global rompe clientes existentes) → Tareas 3, 4, 5.
- Runbook real de despliegue (fuera del alcance de este entorno, sin SSH) → Tarea 15.

Placeholders intencionales (no son plan-failures): `IP_PRIVADA_VPC`, `IP_PUBLICA_VPC`,

`REEMPLAZAR_CON_API_KEY_PUBLICA`, `REEMPLAZAR_CON_CLAVE_HEX_*` — son valores que solo existen una vez se provisionan los droplets reales (IPs asignadas por DigitalOcean) y se generan las claves (Tarea 1, Step 2). Cada tarea que los usa indica explícitamente en qué paso posterior se rellenan (Tarea 15).